

Documentation Technique

Application M-Motors

1. Présentation du projet

L'application permet aux visiteurs de consulter les offres vente et de location de véhicules de la société M-Motors, de créer un compte sécurisé, de créer son

dossier et de télécharger les pièces jointes associées à ce dossier puis de le soumettre. Ils peuvent également choisir quelles options ils souhaitent parmi les 4 proposées dans l'énoncé du sujet, lorsqu'ils souhaitent louer un véhicule.

Un administrateur peut se connecter, gérer les offres de vente et location de véhicule, en rajouter, modifier et supprimer. Il peut également consulter, valider ou refuser tous les dossiers soumis ainsi que les télécharger eux et leurs pièces jointes. Il a également accès à une partie « admin » qui lui ouvre l'entiereté de la BDD en cas de problème.

2. Architecture technique

Le projet repose sur le framework Django en suivant le modèle MVT (Model – View – Template).

Ce modèle sépare la logique métier, la gestion des données et la présentation visuelle.

Le projet a été développé et mis en ligne sur GIT via deux branches : une première branche main sur laquelle le projet a été initialisé, puis une branche fille (develop) de la branche main sur laquelle j'ai développé jusqu'au merge des deux branches à la fin de mon projet.

3. Technologies utilisées

Backend : Django 5.x (Python 3.12)

Base de données : SQLite (développement) / PostgreSQL (production)

Frontend : HTML5, CSS3, Django Template Language

Authentification : Système intégré Django

Sécurité : Hashage PBKDF2, HTTPS, CSRF, etc.

Logs : Sentry (compte gratuit)

4. Modèle conceptuel des données (MCD)

UTILISATEUR

- id_user (PK)
- username
- password
- first_name
- last_name
- email
- (model AbstractUser de base fourni par django)

VEHICULE

- id_vehicule (PK)
- marque
- modele
- motorisation
- annee
- kilometrage
- numero_vin
- prix_vente
- loyer_mensuel
- mode
- statut
- photo
- created_at
- updated_at

OPTION_LOCATION

- id_option (PK)
- code
- label
- description
- prix_mensuel
- is_active

DOSSIER

- id_dossier (PK)
- dossier_type
- statut
- location_duration_months
- location_price
- client_notes
- staff_notes
- created_at
- updated_at
- submitted_at
- decided_at

DOSSIER_DOCUMENT

- id_document (PK)
- document_type
- file
- label
- uploaded_at

DOSSIER_STATUS_HISTORY

- id_history (PK)
- statut_precedent
- nouveau_statut
- commentaire
- changed_at

NOTIFICATION

- id_notification (PK)
- title
- message
- type
- is_read
- created_at

Associations et cardinalités

UTILISATEUR — DOSSIER (Client)

Soumettre

- UTILISATEUR (1,N)
- DOSSIER (1,1)

Un utilisateur peut soumettre plusieurs dossiers.
Un dossier appartient à un seul client.

VEHICULE — DOSSIER

Concerner

- VEHICULE (1,N)
- DOSSIER (1,1)

Un véhicule peut être concerné par plusieurs dossiers.
Un dossier concerne un seul véhicule.

UTILISATEUR — DOSSIER (Agent validateur)

Examiner

- UTILISATEUR (0,N)
- DOSSIER (0,1)

Un agent peut examiner plusieurs dossiers.
Un dossier peut être examiné par un seul agent.

DOSSIER — OPTION_LOCATION

Choisir

- DOSSIER (0,N)
- OPTION_LOCATION (0,N)

Relation N:N matérialisée par la table associative Dossier_location_options.

DOSSIER — DOSSIER_DOCUMENT

Posséder

- DOSSIER (1,1)
- DOSSIER_DOCUMENT (0,N)

Un dossier possède plusieurs documents.
Un document appartient à un seul dossier.

DOSSIER — DOSSIER_STATUS_HISTORY

Historiser

- DOSSIER (1,1)
- DOSSIER_STATUS_HISTORY (0,N)

Un dossier possède plusieurs changements de statut.

UTILISATEUR — DOSSIER_STATUS_HISTORY

Modifier

- UTILISATEUR (0,N)
- DOSSIER_STATUS_HISTORY (0,1)

Un utilisateur peut effectuer plusieurs changements de statut.

UTILISATEUR — NOTIFICATION

Recevoir

- UTILISATEUR (1,1)
- NOTIFICATION (0,N)

Un utilisateur peut recevoir plusieurs notifications.

Une notification est destinée à un seul utilisateur.

5. Sécurité de l'application

- Authentification sécurisée Django avec hashage PBKDF2.
- HTTPS obligatoire et protections natives Django (CSRF, XSS, etc.).
- Accès administrateur restreint et non créable via l'application.
- Gestion des logs administrateurs via Sentry

Les tests sont accessibles dans le fichier tests.py du dossier myapp (il faut commenter la database online et décommenter la database locale apres l'avoir créée sur MySQLWorkbench, sur le fichier settings .py)

6. Évolutions futures

- Intégration d'un module de paiement réel (exemple : Stripe).
- Amélioration du design bootstrap
- Amélioration des notifications

7. Conclusion

Le projet présente une architecture fiable, sécurisée et extensible. Il répond aux besoins du client et ouvre la voie à des évolutions futures, notamment dans la gestion des paiements et la sécurité avancée.